

TD # 2 d'Algo 1 (Vecteurs)

Exercice 1

Ecrire un algorithme permettant de saisir un vecteur T de 10 réels et de trouver et d'afficher le nombre d'occurrences d'un réel R, choisi par l'utilisateur, dans T.

Exemple :

T	2	4	2.5	0	4	5	7	8.8	10	4
---	---	---	-----	---	---	---	---	-----	----	---

Si R = 4 alors le résultat affiché est : 4 apparaît 3 fois dans T. Mais, si R = 12.7 alors le résultat affiché est: 12.7 apparaît 0 fois dans T.

Exercice 2

Soit A un vecteur contenant 10 entiers. Ecrire un algorithme qui permet de vérifier si un élément possède un double dans A. La recherche s'arrête au premier élément pour lequel un double est trouvé. On affichera la position de l'élément ainsi que la position de son double.

Exercice 3

Soient $X = (x_1, x_2, \dots, x_n)$ et $Y = (y_1, y_2, \dots, y_n)$ deux vecteurs. Leur produit scalaire est $P = \sum_{i=1}^n x_i y_i$.

$$\text{Exemple: } \begin{pmatrix} 1 \\ 4 \\ 5 \end{pmatrix} \times \begin{pmatrix} 3 \\ 6 \\ 2 \end{pmatrix} = 1 \times 3 + 4 \times 6 + 5 \times 2 = 37$$

Ecrire un algorithme qui, étant donné une taille de vecteurs n constante (par exemple, $n = 3$), permet de saisir les valeurs de deux vecteurs X et Y de réels, puis d'afficher la valeur du produit scalaire de ces vecteurs.

Exercice 4

Soit un vecteur V de 20 entiers.

Ecrire un algorithme qui permet de :

- Ranger les éléments de V dans un vecteur R de façon à mettre les éléments multiples de 5 de V au début de R suivis des éléments restants.
- Afficher R.
- Afficher la position (indice case) du dernier multiple de 5 stocké dans R (c'est 3 dans l'exemple).

Exemple :

V						
32	7	40	5	12	25	8

R						
40	5	25	32	7	12	8

Exercice 5

Ecrire **un** algorithme qui calcule une somme de produits **SP** à partir de deux tableaux **T1** (de taille **n1**) et **T2** (de taille **n2**). Pour calculer **SP**, il faut multiplier chaque élément du tableau T1 par chaque élément du tableau T2, et additionner le tout. Par exemple si l'on a :

T1

3	6
---	---

T2

4	8	7	12
---	---	---	----

Le **SP** sera :

$$3 * 4 + 3 * 8 + 3 * 7 + 3 * 12 + 6 * 4 + 6 * 8 + 6 * 7 + 6 * 12 = 279$$

Remarque : Traduire en langage Pascal tous les algorithmes proposés.

Les vecteurs en Pascal

1) Syntaxe de la déclaration d'un type vecteur en Pascal:

La déclaration des types non standards (définis par l'utilisateur) se fait en les précédant du mot réservé TYPE.

type <nom_du_vecteur> = **array**[<valmin> ..<valmax>] **of** <type_éléments> ;

exemple : **Tvecteur** = **array**[1..10] **of** **integer** ;

2) Syntaxe de la déclaration d'une variable de type vecteur :

Il y a deux possibilités :

Var

 V : Tvecteur ;

Ou bien

Var

V : array[1..10] **of** **integer** ;

L'élément d'indice i d'un tableau V est désigné par V[i]. Exemple : V[4] := V[2] ;

TD # 2 d'Algo 1 (Vecteurs)

Exercice 1

Algorithme occurrences

Const

N=10

Var

T : tableau (N) de réel

I,nb :entier

R :réel

Debut

{saisie du vecteur}

ecrire ('saisir 10 réels')

Pour i=1 jusqu'à N faire

 Lire (T[i])

Finpour

{calcul de nombre d'occurrence}

ecrire ('donnez la valeur du réel recherché')

Lire(R)

Nb←0

Pour i=1 jusqu'à N faire

 si (T[i]=R) alors

 nb←nb+1

 fini

Finpour

ecrire (R, 'occurre ', nb, ' fois')

Fin

```
program occurrences
```

```
Const
```

```
    N=10 ;
```

```
Var
```

```
    T : array[1..N] of real ;
```

```
    I,nb :integer ;
```

```
    R :real ;
```

```
begin
```

```
    {saisie du vecteur}
```

```
    Writeln ('saisir 10 réels')
```

```
    For i :=1 to N do
```

```
        readln (T[i]) ;
```

```
    {calcul de nombre d'occurrence}
```

```
    Writeln ('donnez la valeur du réel recherché') ;
```

```
    readln(R) ;
```

```
    Nb :=0 ;
```

```
    For i :=1 to N do
```

```
        If (T[i]=R) then
```

```
            nb←nb+1 ;
```

```
    Writeln (R, 'occurre ', nb, ' fois')
```

```
End.
```

Exercice 2

Algorithme elementegaux

Const

N=10

Var

A : tableau (N) de entier

I,j ;pos1,pos2,el :entier

Trouve :booléen

Debut

{saisie du vecteur}

ecrire ('saisir 10 entiers')

Pour i=1 jusqu'à N faire

 Lire (A[i])

Finpour

{Recherche}

trouve ← faux

i ← 1

tant que (i ≤ N-1) et (trouve = faux) faire

 j ← i+1

 tant que (j ≤ N) et (trouve = faux) faire

 si (A[i] = A[j]) alors

 trouve ← vrai

 pos1 ← i

 pos2 ← j

 el ← A[i]

 sinon

 j ← j+1

 finsi

 fintantque

 i ← i+1

Fin tantque

Si (trouve = true) alors

 ecrire (el, 'apparaît 2 fois dans le tableau à la position ', pos1, ' et ', pos2)

sinon

 ecrire ('il n'existe pas 2 éléments égaux dans le tableau')

Finsi

Fin.

```

Program elementegaux ;
Const
  N=10 ;
Var
  A : array[1..N] of integer ;
  I,j,pos1,pos2,el :integer ;
  Trouve :boolean ;

Begin
  {saisie du vecteur}
  writeln ('saisir 10 entiers') ;
  For i :=1 to N do
    readln (A[i]) ;

  {Recherche}
  Trouve :=false ;
  I :=1 ;
  while (i<=N-1) and (trouve=false) do
    begin
      j :=i+1 ;
      while (j<=N) and (trouve=false) do
        if (A[i]=A[j]) then
          begin
            trouve :=true ;
            pos1 :=i ;
            pos2 :=j ;
            el :=A[i] ;
          end
        else
          j :=j+1 ;
        end ;
      i :=i+1 ;
    end ;
  if (trouve =vrai) then
    writeln (el, 'apparaît 2 fois dans le tableau à la position ', pos1, ' et ',pos2)
  else
    writeln ('il n existe pas 2 éléments égaux dans le tableau') ;
  end.

```

Exercice 3

Algorithme produitscalaire

Const

N=10

Var

X,Y : tableau (N) de réel

I :entier

p :réel

Debut

{saisie du vecteur X}

ecrire ('saisir ',N, ' réels')

Pour i=1 jusqu'à N faire

 Lire (X[i])

Finpour

{saisie du vecteur y}

ecrire ('saisir 10 réels')

Pour i=1 jusqu'à N faire

 Lire (Y[i])

Finpour

{calcul du produit scalaire}

P←0

Pour i=1 jusqu'à N faire

 P←P+X[i]*Y[i]

Finpour

ecrire ('Le produit scalaire est égal à ', P)

Fin

```
program produitscalaire;
```

```
Const
```

```
    N=10;
```

```
Var
```

```
    X,Y : array[1..N] of real;
```

```
    I :integer;
```

```
    p :real;
```

```
begin
```

```
    {saisie du vecteur X}
```

```
    writeln('saisir ',N, ' réels');
```

```
    for i:=1 to N do
```

```
        readln (X[i]);
```

```
    {saisie du vecteur y}
```

```
    writeln('saisir 10 réels');
```

```
    for i:=1 to N do
```

```
        readln (Y[i]);
```

```
    {calcul du produit scalaire}
```

```
    P:=0;
```

```
    for i:=1 to N do
```

```
        P:=P+X[i]*Y[i];
```

```
    writeln('Le produit scalaire est égal à ', P:2:2);
```

```
end.
```

Exercice 4

Algorithme multiple5

Const

N=20

Var

V,R : tableau (N) de entier

I,j,pos :entier

Debut

{saisie du vecteur V}

ecrire ('saisir ',N, ' entiers')

Pour i=1 jusqu'à N faire

Lire (V[i])

Finpour

{construction du vecteur R}

j←1

Pour i=1 jusqu'à N faire

Si (V[i] MOD 5 =0) alors

R[j] ←V[i]

j←j+1

finsi

Finpour

Pos ←j-1 ;

Pour i=1 jusqu'à N faire

Si (V[i] MOD 5 <>0) alors

R[j] ←V[i]

j←j+1

finsi

Finpour

{Affichage de R}

Pour i=1 jusqu'à N faire

Ecrire (R[i])

Finpour

ecrire ('Le dernier multiple de 5 se trouve à la position ', Pos)

Fin

```

program multiple5;
Const
  N=20;

Var
  V,R : array[1..n] of integer;
  I,j,pos :integer;

begin
  {saisie du vecteur V}
  writeln ('saisir ',N, ' entiers');
  for i:=1 to N do
    readln (V[i]);

  Writeln;
  {construction du vecteur R}
  j:=1;
  for i:=1 to N do
    if (V[i] MOD 5 =0) then
      begin
        R[j] :=V[i];
        j:=j+1;
      end;

  Pos:=j-1 ;
  for i:=1 to N do
    if (V[i] MOD 5 <>0) then
      begin
        R[j] :=V[i];
        j:=j+1;
      end;

  {Affichage de R}
  for i:=1 to N do
    writeln (R[i]);

  writeln ('Le dernier multiple de 5 se trouve à la position ', Pos);
end.

```

Exercice 5

Algorithme produit

Const

N=3

M=5

Var

T1 : tableau (N) de entier

T2 : tableau (M) de entier

I,j,p:entier

Debut

{saisie du vecteur T1}

ecrire ('saisir ',N, ' entiers')

Pour i=1 jusqu'à N faire

 Lire (T1[i])

Finpour

{saisie du vecteur T2}

ecrire ('saisir ',M, ' entiers')

Pour i=1 jusqu'à M faire

 Lire (T2[i])

Finpour

{calcul de la somme du produit }

P←0

Pour i=1 jusqu'à N faire

 Pour j=1 jusqu'à M faire

 P←P+T1[i]*T2[j]

 Finpour

finpour

ecrire ('La somme du produit est égal à ', P)

Fin

```
program produit;
```

```
Const
```

```
    N=3;
```

```
    M=5;
```

```
Var
```

```
    T1 : array [1..N] of integer;
```

```
    T2 : array [1..M] of integer;
```

```
    I,j,p:integer;
```

```
begin
```

```
    {saisie du vecteur T1}
```

```
    writeln ('saisir ',N, ' entiers');
```

```
    for i:=1 to N do
```

```
        readln (T1[i]);
```

```
    {saisie du vecteur T2}
```

```
    writeln ('saisir ',M, ' entiers');
```

```
    for i:=1 to M do
```

```
        readln (T2[i]);
```

```
    {calcul de la somme du produit }
```

```
    P:=0;
```

```
    for i:=1 to N do
```

```
        for j:=1 to M do
```

```
            P:=P+T1[i]*T2[j];
```